

Autonome Eingebettete Systeme (AEiSy)

Level 3 – Embedded Software Architektur

1. Entwicklungsprozess
2. Architektur
3. Codierung / Implementierung

Fakultät Elektrotechnik und Informatik

Prof. Dr. Markus Hager

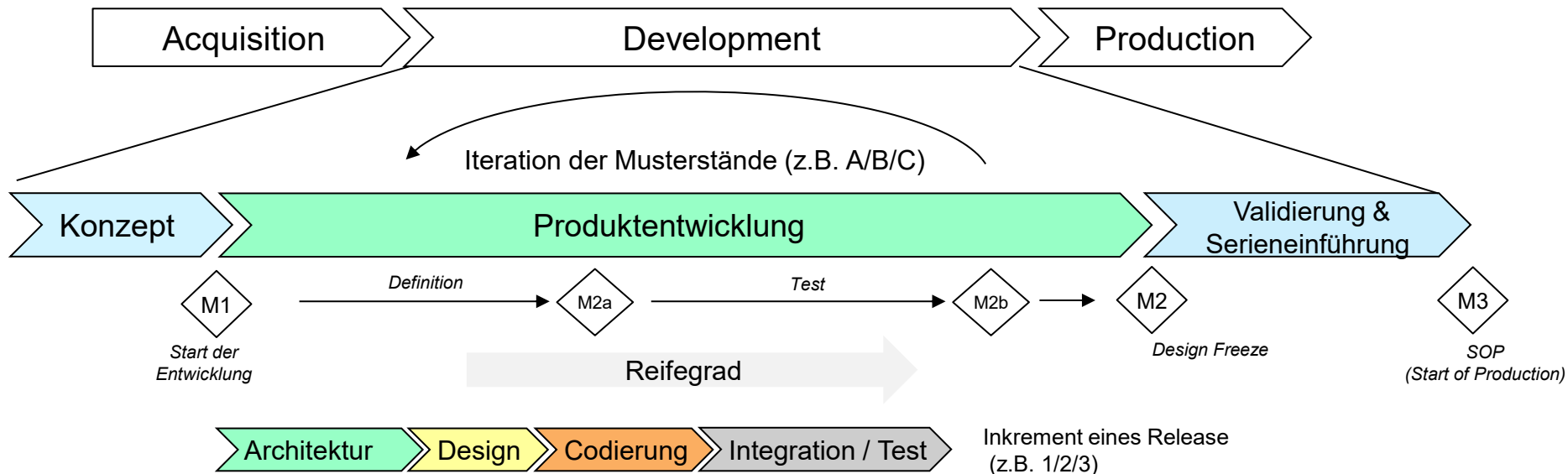
Level 3: Embedded Software Architektur

Lernziele:

- Strukturierung der Software (→ Reduktion der Komplexität)
- Kennen und Anwenden von Coding Guidelines
- Kennen und Anwenden von Design Patterns (= Entwurfsmuster)
- Kennen und Anwenden eines Entwicklungsprozesses

1. SW-Entwicklungsprozess

- beschreibt den zeitlichen Verlauf von Problemstellung über Realisierung bis zum Einsatz einer Software.
- Phasenmodell: (Beispiel aus Sicht eines Zulieferers / Steuergeräteentwicklung)



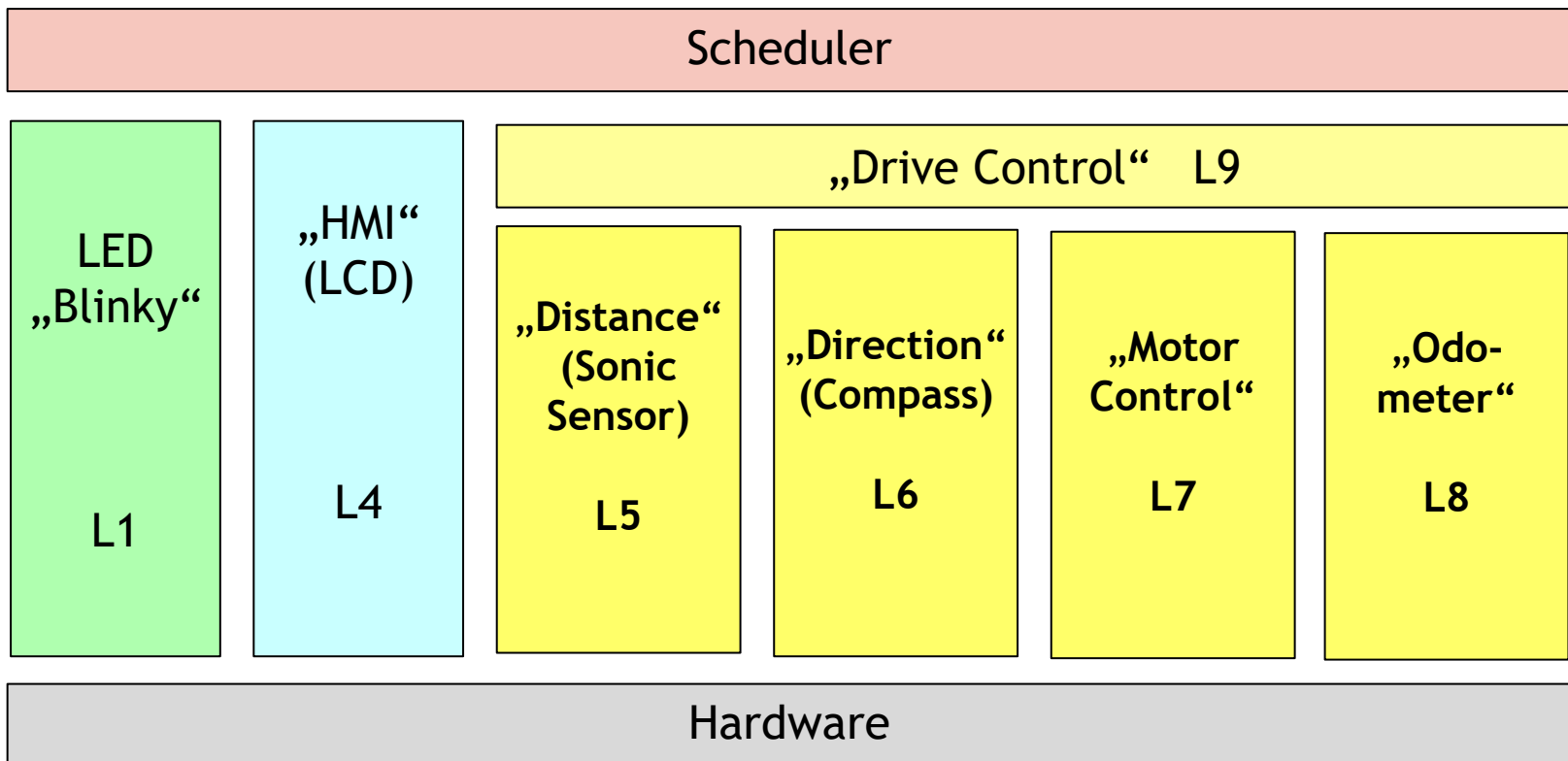
→ siehe auch Vorgehensmodelle: z.B. V-Modell, Wasserfallmodell ...

2. SW-Architektur

Architektur: Grobentwurf (Skizze, Struktur) – Wie ist die SW aufgebaut?

Ziel: Übersicht, Reduktion der Komplexität

Beispiel: Funktionsübersicht, Schichtenmodell

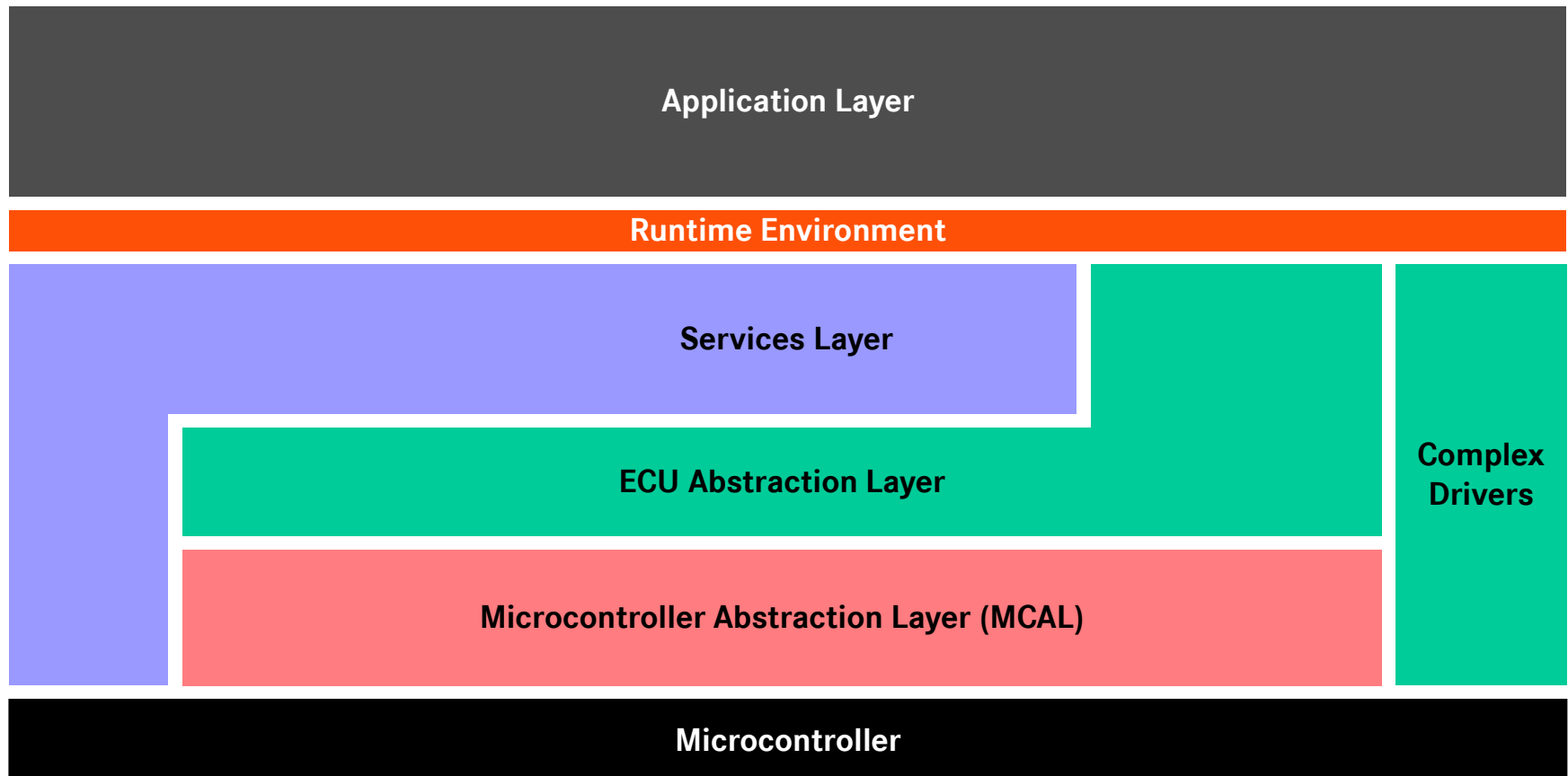


2. SW-Architektur

Standardarchitektur: Architekturmuster

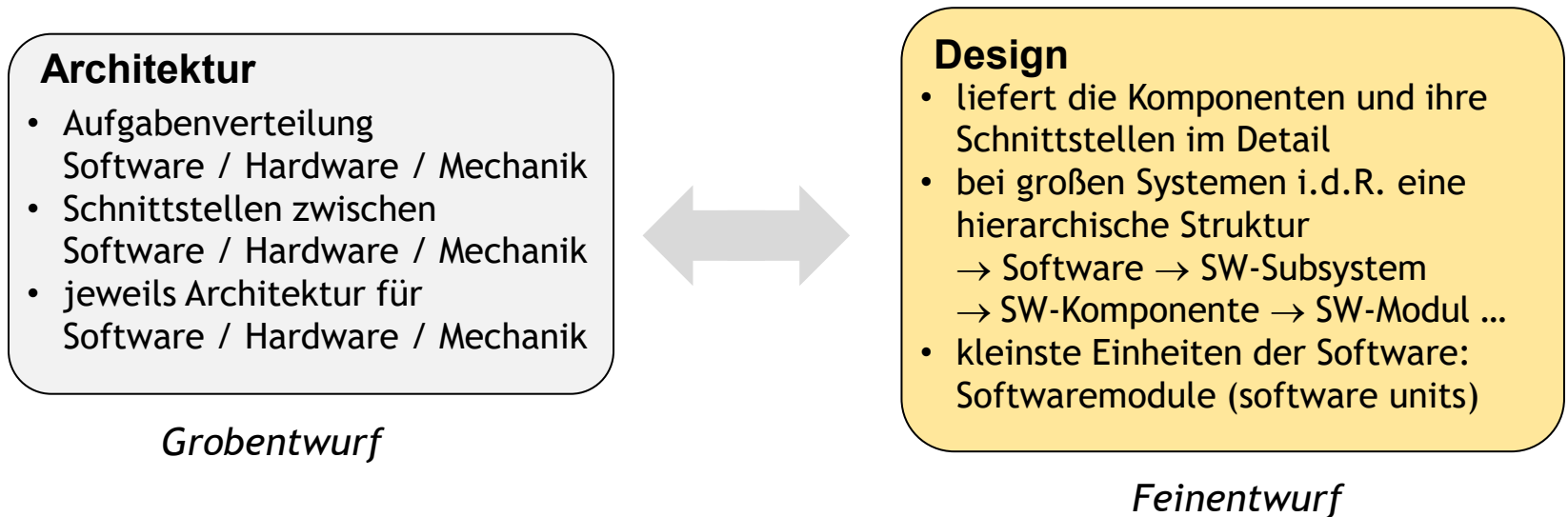
AUTOSAR

- Bsp. Layerarchitektur / Schichtenmodell (vgl. Autosar) AUTomotive Open System ARchitecture



2. SW-Architektur

Design: Feinentwurf – Wie funktioniert die SW im Detail?

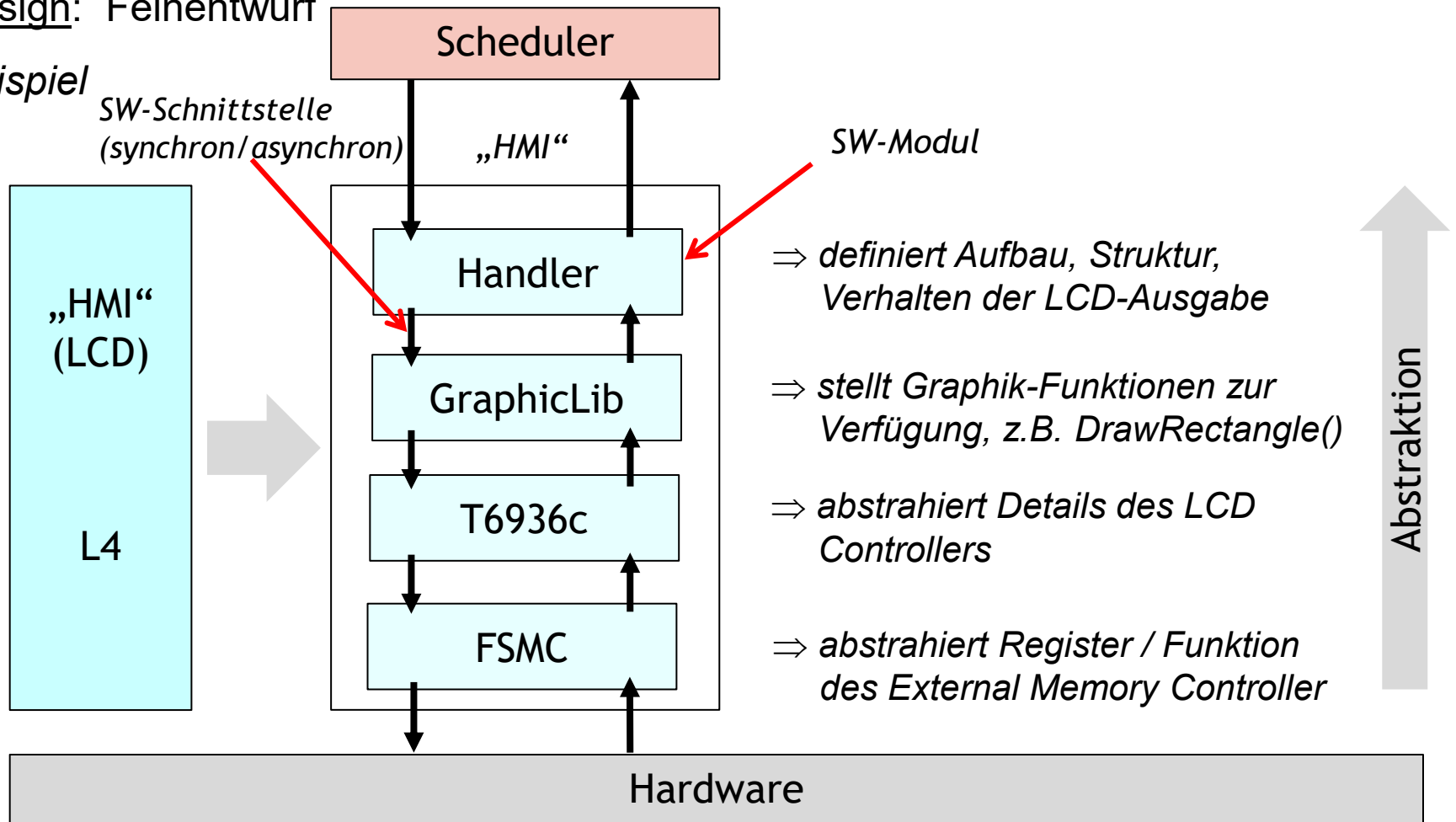


⇒ Das Design beschreibt die Details der Schichten der Architektur.

2. SW-Architektur

Design: Feinentwurf

Beispiel

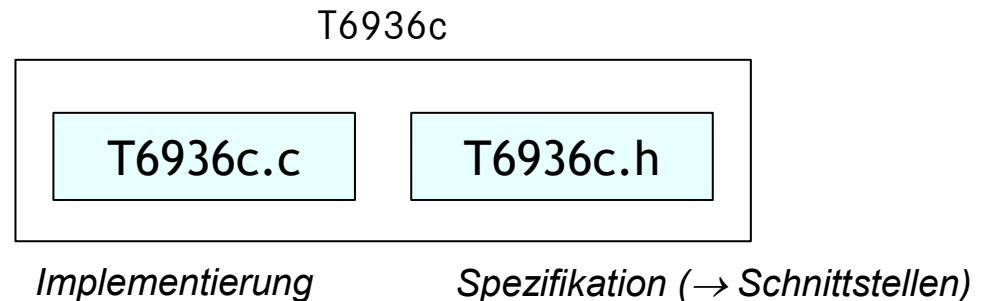


3. Codierung / Implementierung

Modularisierung:

- Ein SW – Modul beinhaltet alle Funktionen und deren verwendeten Daten (oder HW in Form von Register).
- ⇒ Vergleich: Klasse in objektorientierten Sprachen
- Ein SW – Modul besteht aus einer Quellcodedatei (*.c) und einer Headerdatei (*.h).

Beispiel:



- Vorteile der Modularisierung: Portierbarkeit, Lesbarkeit, Wiederverwendbarkeit

Typischer Aufbau eines SW - Moduls

- Quellcodedatei:

```
#include "modul.h"

// Modulvariablen
static int var1;

// Funktionsprototypen
static void func1(void);

// private Funktionen
static void func1(void)
{
}

// öffentliche Funktionen
// = Schnittstellen des Moduls
void func2(void)
{
}
```

→ Einbinden der eigenen Headerdatei (aber auch weitere)

→ Schlüsselwort **static**
⇒ Variable nur innerhalb des Moduls sichtbar (Kapselung)

→ modulinterne (*private*) Funktionen werden mit dem Schlüsselwort **static** deklariert/definiert (Prototyp + Implementierung).

→ öffentliche (*public*) Funktionen werden hier implementiert (kein **static**, Prototyp in Headerdatei).

Typischer Aufbau eines SW - Moduls

- Headerdatei:

```
#ifndef _MODUL_h_
#define _MODUL_h_

#include "typedefs.h"

// Typdefinitionen
typedef int int32_t;

// Prototypen der
// öffentlichen Funktionen
void func2(void);

#endif
```

modul.h

→ verhindert das mehrmalige Einbinden von Headerdateien!

→ **#include** sollte in Headerdateien vermieden werden
Ausnahme: Headerdatei mit eigenen Typdefinitionen

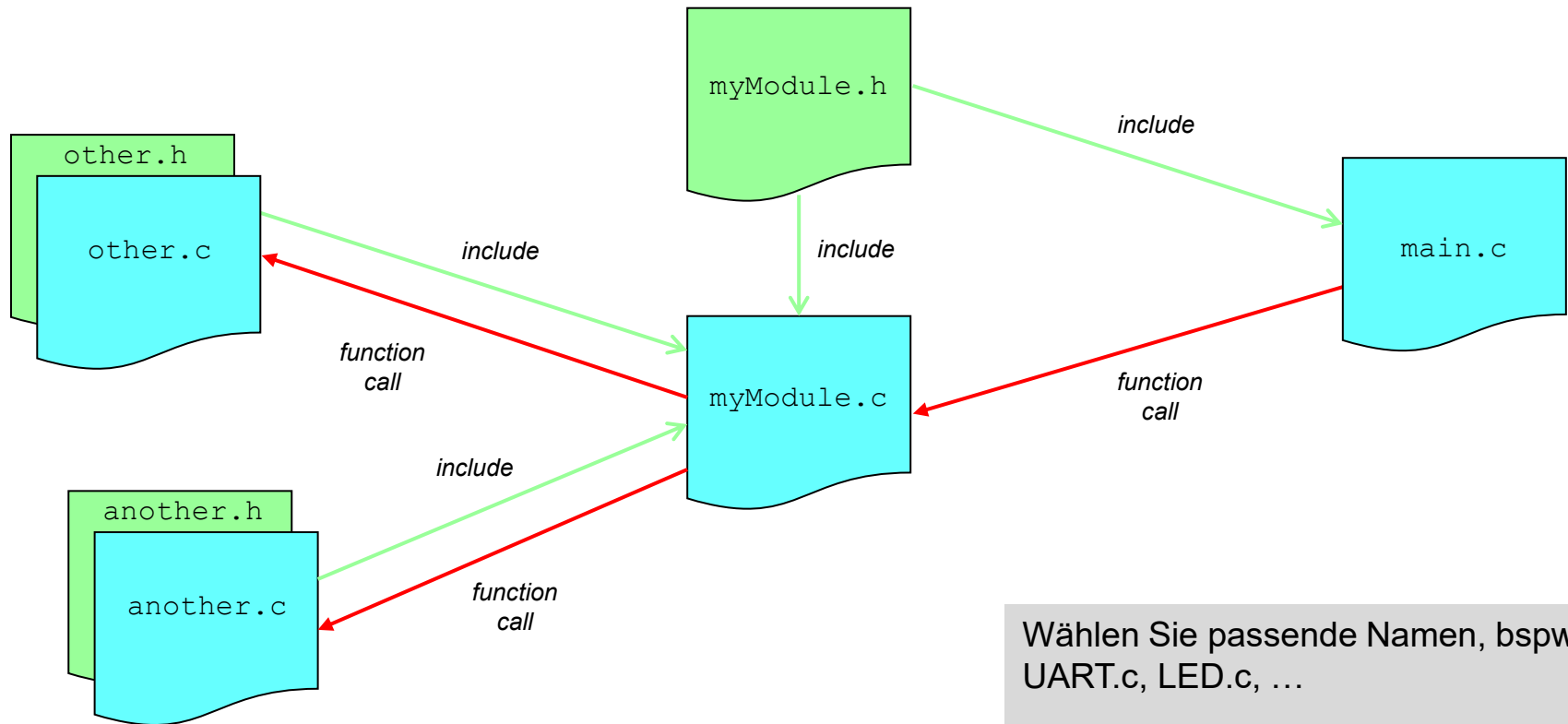
→ weitere Typdefinitionen

→ Veröffentlichung der Schnittstellenfunktionen des Moduls;
Implementierung der Funktion in der *.c Datei

→ siehe auch Templates in Moodle!

Einbinden eines SW - Moduls

- `#include` Hierarchie

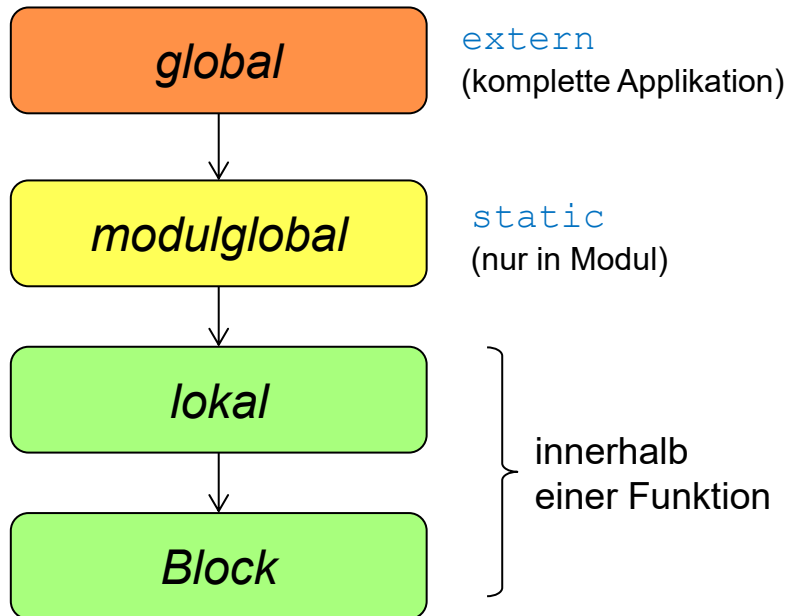


Wählen Sie passende Namen, bspw. UART.c, LED.c, ...

Pi-mal-Daumen Regel: Eine Datei max. ca. 200 Zeilen, ab 500 Zeilen wird es kritisch!

Der Scope einer Variablen

- Das Risiko, dass auf eine Variable oder Funktion fälschlicherweise zugegriffen wird, steigt mit der Größe des Geltungsbereichs (= *Scope*) der Variable oder Funktion.
 - ⇒ Scope einer Variablen so klein wie möglich
 - ⇒ vermeide (applikations-) globale Variablen!



```
void func( int i )
{
    ....
    { // Block
        int i;
        for ( i = 0; i < 10; i++ )
            ...;
    }
    ....
}
```

„Scheduler“

- Synchronisation zwischen Interrupt und Hauptprogramm

→ typische Implementierung:

```
extern uint8_t  flag;

void EXTI0_IRQHandler (void)
{
    flag = 1;
}
```

isr.c

- Beachte: Die flag Variable sollte/muss **volatile** deklariert sein!

```
volatile uint8_t  flag = 0;

int main( void )
{
    while ( 1 )
    {
        if( flag == 1 )
        {
            flag = 0;

            /* Code für Task A*/
            ...
        }
    }
}
```

main.c

- Problem: Variable „flag“ muss hier global deklariert werden!

„Scheduler“

- besser:

```
int main( void )
{
    while ( 1 )
    {
        Handler();
    }
}
```

main.c

```
void EXTI0_IRQHandler (void)
{
    SetFlag();
}
```

isr.c

include

include

```
#ifndef _MODUL_h_
#define _MODUL_h_

#include "typedefs.h"

void Handler( void );
void SetFlag( void );

#endif
```

```
void Handler( void )
{
    if ( flag == 1 )
    {
        flag = 0;

        /* Code für Task A*/
        ...
    }
}

...

void SetFlag( void )
{
    flag = 1;
}
```

modul.c

Eventmechanismus

- Kommunikation / Datenaustausch zwischen den Handlern:

```
typedef struct {  
    uint16_t  EventID;  
    uint16_t  EventParameter;  
} EVENT_T;
```

event.h

```
int main( void )  
{  
    EVENT_T curEvent;  
    ...  
    SetEvent(EVT_INIT, 0);  
    while ( 1 )  
    {  
        curEvent = GetEvent();  
        Handler1(curEvent);  
        Handler2(curEvent);  
        ...  
    }  
}
```

main.c

```
void SysTick_Handler(void)  
{  
    static uint32_t cnt = 0;  
  
    SetEvent(EVT_10MS, 0);  
}
```

isr.c

EVENT

```
void Handler1(EVENT_T curEvent )  
{  
    switch (curEvent.EventID)  
    {  
        case EVT_NOEVENT  
            break;  
  
        case EVT_INIT:  
            initHandler();  
            break;  
        ...  
    }  
  
    switch( state )  
    {  
        case 0:  
            break;  
        ...  
    }  
}
```

modul.c

```
typedef struct {  
    uint16_t  EventID;  
    uint16_t  EventParameter;  
} EVENT_T;
```

event.h

Eventmechanismus

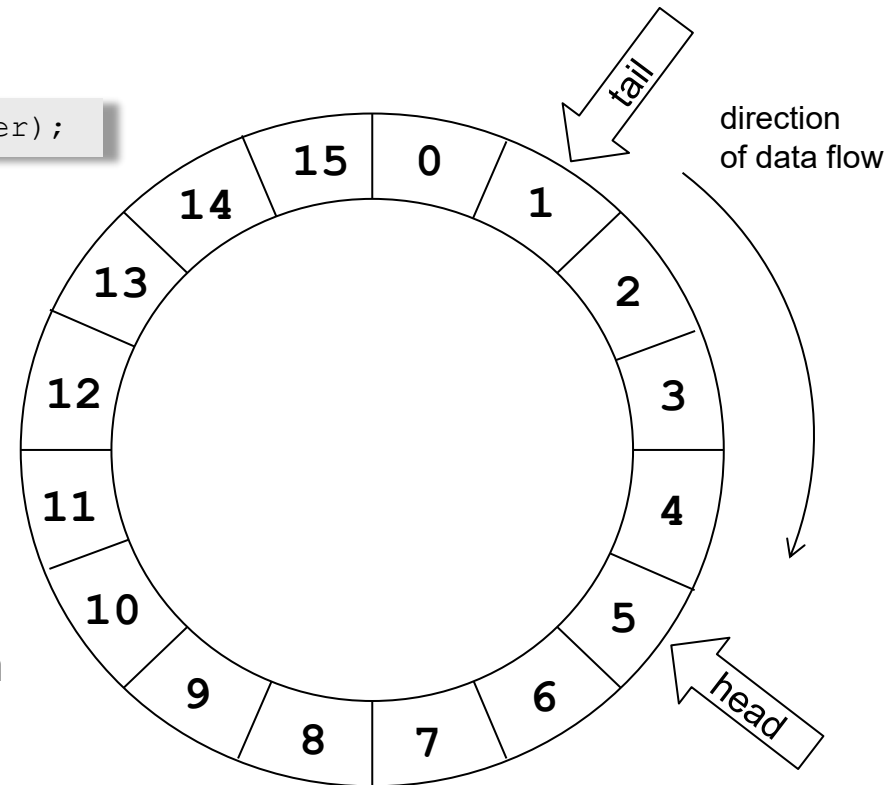
- Ringbuffer als gemeinsame Datenstruktur zur Übergabe eines Events
- FiFo – Semantik (first-in-first-out)
- Implementierung als Array (→ Überlauf / Ende beachten)
- Schnittstellen:

```
void SetEvent(uint16_t eventID, uint16_t Parameter);
```

- neues Event an aktuelle Schreibposition (head Zeiger)

```
EVENT_T GetEvent( void );
```

- „ältestes“ Event aus Queue lesen (tail Zeiger)
- Abstand von head und tail entspricht dem Füllstand des Buffers



Projektstruktur (Beispiel)

Systemstart

System / Mikrocontroller

Hilfsfunktionen

Hardwareabstraktion

Applikation / Funktionen

